

Báo cáo Nghiên cứu Khoa học: Hiện thực hóa các hàm logic bất kỳ sử dụng bộ giải mã (Decoder) và cổng OR phục vụ tổng hợp logic tự động

Thông tin nhóm

- **Lớp:** SE1930
 - **Nhóm:** 6
 - **Trưởng nhóm:** Phạm Lê Nhật Minh - SE193155
 - **Thành viên:**
 - Phan Trí Dũng - SE193224
 - Trần Minh Nhật - SE193149
 - Lê Nguyễn Hoàng Lộc - SE193098
-

Tóm tắt (Abstract)

<Phần này cần được viết để tóm tắt toàn bộ công trình, bao gồm vấn đề, phương pháp, kết quả chính và đóng góp.>

Từ khóa (Keywords): Logic Synthesis, Decoder, Combinational Logic, Combinational Mạch, VLSI.

1. Giới thiệu (Introduction)

1.1. Bối cảnh và Vấn đề (Problem Statement)

Trong thiết kế logic số truyền thống, việc hiện thực hóa các hàm Boolean tùy ý chủ yếu dựa vào các mạng lưới cổng logic cơ bản (AND, OR, NOT) được thiết kế riêng và không có cấu trúc đồng nhất. Khi độ phức tạp của các hệ thống số tăng lên, quy trình định tuyến thủ công này trở nên phức tạp theo cấp số nhân và rất dễ xảy ra lỗi do con người. Việc thiếu một layout cấu trúc được tiêu chuẩn hóa khiến cho phần cứng khó ánh xạ, sửa đổi và mở rộng.

Tầm quan trọng của vấn đề này ngày càng trở nên rõ rệt khi các hệ thống bán dẫn hiện đại và kiến trúc VLSI (ví dụ: phần cứng cho Học máy, Mạng nơ-ron nhị phân hóa) tiếp tục mở rộng. Các mạng cổng không đều gây ra độ trễ lan truyền không thể đoán trước và tắc nghẽn định tuyến. Hơn nữa, các cổng rời rạc không được tối ưu hóa sẽ tiêu thụ diện tích silicon và năng lượng quá mức. Quan trọng hơn, việc thiếu một cấu trúc topo module, được tiêu chuẩn hóa đã tạo ra một nút thắt cổ chai, ngăn cản các công cụ Tự động hóa Thiết kế Điện tử (EDA) tự động hóa hoàn toàn quy trình tổng hợp logic một cách đáng tin cậy.

1.2. Động lực nghiên cứu (Motivation)

Việc tìm ra một phương pháp tiếp cận có cấu trúc, module hóa—chẳng hạn như sử dụng Bộ giải mã (Decoders) kết hợp với cổng OR—là rất quan trọng để hợp lý hóa quy trình tổng hợp phần cứng tự động và tối ưu hóa tài nguyên vật lý. Việc tiêu chuẩn hóa việc hiện thực logic bằng kiến trúc Decoder-OR không chỉ tối ưu hóa tài nguyên phần cứng vật lý mà còn cho phép xác minh bằng thuật toán và mở đường cho các quy trình tổng hợp logic hoàn toàn tự động, có thể đọc được bằng máy.

1.3. Lĩnh vực ứng dụng (Application Domain)

Nghiên cứu này thuộc các lĩnh vực Tự động hóa Thiết kế Điện tử (EDA), Thiết kế Vi mạch Tích hợp Cỡ Rất Lớn (VLSI), và Thiết kế Mạch Logic Tổ hợp. Đối tượng người dùng mục tiêu bao gồm Kỹ sư Phần cứng, Nhà thiết kế Mạch số, Nhà phát triển Công cụ EDA và các nhà nghiên cứu Kiến trúc VLSI.

2. Tổng quan các công trình liên quan (Related Work)

2.1. Các hướng tiếp cận trước đây

Nghiên cứu gần đây đã tiếp cận việc tổng hợp logic và tối ưu hóa bộ giải mã từ ba góc độ riêng biệt: - **Xác minh Thuật toán:** Ikwunne & Okide (2020) đã phát triển các thuật toán tương tác để xác minh bằng toán học sự tương đương giữa các biểu thức Boolean phức tạp và các phương trình đã được đơn giản hóa, loại bỏ lỗi của con người trong việc đánh giá bảng chân lý. - **Tối ưu hóa Phần cứng:** Balobas & Konofaos (2025) đã tối ưu hóa các bộ giải mã CMOS ở cấp độ transistor bằng cách sử dụng Logic Ba-Transistor (3TL), giúp giảm đáng kể công suất hoạt động/chờ và độ trễ lan truyền so với CMOS tĩnh thông thường. - **Phân rã Boolean:** Costamagna & De Micheli (2023) đã đề xuất thuật toán Phân rã Hỗ trợ Rời rạc Nhận biết “Không quan tâm” (DK-DSD) để phân vùng các biến một cách có hệ thống và tổng hợp các hàm Boolean được chỉ định một phần, phục hồi độ chính xác chức năng cho các mô hình Học máy.

2.2. Hạn chế và Khoảng trống nghiên cứu

Mặc dù có những tiến bộ này, một số khoảng trống quan trọng vẫn còn tồn tại: - **Sự mất kết nối giữa Xác minh và Tổng hợp Phần cứng:** Các thuật toán xác minh hiện có hoàn toàn mang tính toán học; chúng không tự động dịch các minterm đã được xác minh thành một lược đồ định tuyến vật lý có thể đọc được bằng máy. - **Chi phí tính toán trong Phân rã:** Các thuật toán phân rã tiên tiến như DK-DSD có độ phức tạp tính toán cao ($O(n^2N)$) và phụ thuộc nhiều vào các mạch xấp xỉ bên ngoài để hoạt động, khiến chúng khó ánh xạ trực tiếp vào phần cứng có cấu trúc mà không gặp tắc nghẽn. - **Tích hợp Phẳng và Phân cấp:** Mặc dù các tối ưu hóa ở cấp độ transistor thấp (như 3TL) đã tồn tại, nhưng vẫn thiếu các quy trình làm việc ở cấp độ cao để tự động tích hợp các khối giải mã hiệu quả này vào các mạng cổng OR đa tầng cho các biểu thức Boolean không đều.

Khoảng trống nghiên cứu chính: Các phương pháp tổng hợp logic hiện tại chủ yếu tập trung vào tối ưu hóa transistor ở cấp thấp hoặc phân rã toán học nặng về tính toán. Rất ít sự chú ý được dành cho việc tạo ra một quy trình hoàn toàn tự động, có thể đọc bằng máy (ví dụ: lược đồ JSON) để chuyển đổi trực tiếp các minterm Boolean đã được xác minh thành các cấu hình phần cứng Decoder-OR được tối ưu hóa về mặt cấu trúc.

2.3. Bảng tổng hợp (Literature Review Matrix)

Criteria	Paper 1	Paper 2	Paper 3
Paper Title	Accuracy recovery: A decomposition procedure for the synthesis of partially-specified Boolean functions	Optimization of CMOS Decoders Using Three-Transistor Logic	Interactive Algorithms for the Verification of the Equality between Complex and Simplified Boolean-Algebra Expressions in Digital Decoders
Year	2023	2025	2020

Criteria	Paper 1	Paper 2	Paper 3
Venue	INTEGRATION, the VLSI Journal	MDPI Electronics	Review of Computer Engineering Research
Domain	Logic Synthesis / ML Hardware	VLSI / CMOS Decoders	Boolean Verification
Core Method	Don't Knows-Aware Disjoint Support Decomposition (DK-DSD)	3-Transistor Logic (3TL)	Interactive C++ verification algorithm
Dataset	IWLS2020 benchmarks, binary MNIST	15 nm FinFET Predictive Models	BCD to 7-Segment truth tables
Metrics	Test Accuracy, Hardware Size, Circuit Depth, Runtime	Active/Standby Power, Delay, Transistor Count	Output matching accuracy
Main Contribution	Reduced gates by 17.38%, circuit depth by 12.02%, and achieved 95.73% accuracy on MNIST.	Significant reduction in transistor count, power consumption, and propagation delay.	Automated mathematical verification of complex vs simplified logic equivalence.
Limitation	High computational complexity ($O(n^2N)$); relies on initial approximate circuits.	Lacks hierarchical optimization for multi-stage OR-gate networks.	Purely algorithmic; ignores physical hardware synthesis and routing.
Relevance to Our Topic	Provides a mathematical framework for partitioning input variables and reducing redundant routing (RQ2.2).	Provides hardware-level evidence of decoder efficiency vs discrete gates (RQ2.3).	Validates theoretical relationship between minterms and decoder outputs (RQ2.1).

3. Câu hỏi nghiên cứu (Research Questions)

Để giải quyết vấn đề đã xác định và lấp đầy khoảng trống nghiên cứu, dự án này được định hướng bởi một câu hỏi nghiên cứu chính và ba câu hỏi phụ:

Câu hỏi chính: RQ-02: Làm thế nào một bộ giải mã có thể được sử dụng kết hợp với các cổng OR để hiện thực hóa một hàm logic thay vì sử dụng các cổng logic cơ bản?

Câu hỏi phụ: - RQ2.1: Mối quan hệ toán học và thuật toán giữa các đầu ra của một Bộ giải mã và các minterm của một hàm logic là gì? - **RQ2.2:** Đối với một hàm Boolean cho trước, làm thế nào để chọn một cách có hệ thống các đầu ra chính xác của Bộ giải mã để kết nối với cổng OR, và làm thế nào việc định tuyến này có thể được tự động hóa (ví dụ: thông qua lược đồ JSON)? - **RQ2.3:** Những lợi thế về cấu trúc và phần cứng (ví dụ: số lượng transistor, tính module, công suất hoạt động/chờ) của việc hiện thực logic bằng Bộ giải mã so với thiết kế bằng các mạng cổng AND/OR/NOT rời rạc là gì?

4. Phương pháp đề xuất (Proposed Method)

5. Thiết kế hệ thống (System Design)

5.1. Các tính năng của hệ thống (System Features)

1. Phân tích các phương trình Boolean đầu vào và xác minh sự tương đương toán học của chúng với các biểu thức minterm chính tắc.
2. Tự động phân rã các biến để ánh xạ phần cứng hiệu quả.
3. Ánh xạ các minterm đã xác định một cách nghiêm ngặt tới các chân đầu ra phần cứng tương ứng của một Bộ giải mã.
4. Tạo ra một lược đồ định tuyến JSON được tiêu chuẩn hóa, có thể đọc được bằng máy, quy định các kết nối vật lý giữa các đầu ra của Bộ giải mã và các đầu vào của cổng OR.

5.2. Lược đồ đầu ra (Output Schema)

<Phần này có thể trình bày chi tiết về cấu trúc JSON schema đã được định nghĩa trong file `system_output_schema.json`.>

5.3. Case Study: Mạch giải mã BCD sang hiển thị LED 7 đoạn

Để làm rõ tính trực quan và ưu việt của hệ thống tổng hợp tự động, nghiên cứu sử dụng bài toán hiển thị một chữ số thập phân cụ thể — ví dụ là số 5 — trên LED 7 đoạn làm minh họa. Theo cách thiết kế truyền thống, để hiển thị số 5, kỹ sư phải tính toán tối giản hàm cho cả 7 đoạn độc lập và đi dây thủ công một mạng lưới phức tạp gồm nhiều cổng AND, NOT rời rạc đan chéo nhau. Với kiến trúc Decoder-OR tự động, quy trình định tuyến này được tối ưu hóa theo thực nghiệm như sau:

- **Bước 1 (Phân tích Toán học):** Chữ số 5 trong hệ thập phân khi chuyển sang mã BCD 4-bit sẽ có giá trị là 0101, tương ứng với minterm m_5 . Để cấu thành nên hình ảnh số 5 trên mặt hiển thị, các phân đoạn LED cần phải được kích hoạt đồng thời bao gồm: a, c, d, f, g .
- **Bước 2 (Ánh xạ Phần cứng):** Hệ thống sử dụng một Bộ giải mã 4-sang-16 nhận 4 bit đầu vào (A, B, C, D). Do đặc tính phần cứng đã được cấu trúc định sẵn, chân ngõ ra Y_5 của bộ giải mã sẽ là chân duy nhất được kích hoạt lên mức 1 khi và chỉ khi có tổ hợp mã BCD của số 5 đầu vào.
- **Bước 3 (Định tuyến Tự động):** Script của hệ thống sẽ tự động quét trạng thái và sinh ra một file JSON cấu hình định tuyến phần cứng: kết nối trực tiếp chân ngõ ra Y_5 của bộ giải mã làm một trong các đầu vào của 5 cổng OR độc lập phụ trách các đoạn a, c, d, f, g .

Quy trình ánh xạ tự động này được áp dụng đồng thời cho tất cả các chữ số từ 0 đến 9 trên cùng một bộ giải mã dùng chung. Khi file JSON thực thi cấu hình nối dây, mạng lưới kết nối giữa các chân Decoder (Y_0 đến Y_9) và 7 cổng OR của các phân đoạn hiển thị được thiết lập tự động, tạo nên một hệ thống có cấu trúc module đồng nhất, giảm thiểu tối đa diện tích transistor và tối ưu hóa đường đi của tín hiệu trên layout.

6. Kế hoạch đánh giá (Evaluation Plan)

- **Tập dữ liệu (Dataset):** Các mạch tổ hợp tiêu chuẩn (ví dụ: bộ cộng toàn phần 1-bit, bộ giải mã BCD sang hiển thị 7 đoạn).
- **Phương pháp đối chứng (Baseline):** Các hiện thực logic truyền thống sử dụng các cổng logic cơ bản rời rạc (AND/OR/NOT).

- **Các chỉ số đo lường (Metrics):** Các chỉ số hiệu quả phần cứng bao gồm số lượng transistor, số lượng cổng (độ phức tạp cấu trúc), và tính module.
 - **Đánh giá của chuyên gia (Expert evaluation):** Xác minh tính đúng đắn của logic định tuyến JSON tự động bởi các giảng viên VLSI hoặc chuyên gia trong lĩnh vực.
 - **Khảo sát người dùng (User survey):** Không áp dụng (Việc đánh giá tập trung hoàn toàn vào các chỉ số phần cứng định lượng và độ chính xác toán học).
-

7. Đóng góp dự kiến (Expected Contributions)

1. Một quy trình tự động, được tiêu chuẩn hóa để dịch bất kỳ hàm Boolean nào thành một cấu trúc phần cứng Decoder-OR.
 2. Xác minh bằng thuật toán và lý thuyết chứng minh việc ánh xạ chính xác các minterm tới các đầu ra của bộ giải mã, loại bỏ các lỗi định tuyến do con người.
 3. Chứng minh hiệu quả phần cứng (ví dụ: giảm số lượng transistor và tính module cấu trúc) của kiến trúc được đề xuất so với các mạng cổng rời rạc truyền thống.
-

8. Thảo luận (Discussion)

<Phần này dành để thảo luận về kết quả, ý nghĩa của chúng, các hạn chế và hướng phát triển trong tương lai.>

9. Kết luận (Conclusion)

<Phần này tóm tắt lại các điểm chính của bài nghiên cứu và khẳng định lại các đóng góp.>

Tài liệu tham khảo (References)

1. A. Costamagna, G. De Micheli, "Accuracy recovery: A decomposition procedure for the synthesis of partially-specified Boolean functions," INTEGRATION, the VLSI journal, 2023. <https://www.science.org>
2. D. Balobas, N. Konofaos, "Optimization of CMOS Decoders Using Three-Transistor Logic," MDPI Electronics, 2025. <https://www.mdpi.com/2079-9292/14/5/914>
3. T. A. Ikwunne, S. O. Okide, "Interactive Algorithms for the Verification of the Equality between Complex and Simplified Boolean-Algebra Expressions in Digital Decoders," Review of Computer Engineering Research, 2020. <https://archive.conscientiabeam.com/index.php/76/article/view>